OPERATING SYSTEMS

25CS2102

LAB WORKBOOK

STUDENT ID:
STUDENT NAME:
ACADEMIC YEAR: 2026-27

Table of Contents

1. Session 01: Essential linux commands 01
2. Session 02 Basic file and process subsystem system calls 12
3. Session 03: System calls 22
4. Session 04: Multithreading programs 34
5. Session 05: Cpu scheduling algorithms 44
6. Session 06: Proc file system.....53
7. Session 07: System V Shared Memory.....68
8. Session 08: Kernel Buffering of File I/O.....79
9. Session 09: File subsystem system calls and file commands 93
10. Session 10: Mutex and condition variables 107
11. Session 11: Classic synchronization problems 119
12. Session 12: Bpf() system call.....131
13. Session 13: Apply bpftrace as the awk-of-the-kernel.....146

A.Y. 2026-27 LAB/SKILL CONTINUOUS EVALUATION

S.No	Date	Experiment Name	Pre-Lab (10M)	In-Lab (25M)			Post-Lab (10M)	Viva Voce (5M)	Total (50M)	Faculty Signature
				Program/ Procedure (5M)	Data and Results (10M)	Analysis & Inference (10M)				
1.		Essential linux commands								
2.		Basic file and process subsystem system calls								
3.		System calls								
4.		Multithreading programs								
5.		Cpu scheduling algorithms								
6.		Proc file system								
7.		System V Shared Memory								
8.		Kernel Buffering of File I/O								
9.		File subsystem system calls and file commands								
10.		Mutex and condition variables.								
11.		Classic synchronization problems								
12		Bpf() system call								
13.		Apply bpftrace as the awk-of-the-kernel								

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: LINUX Commands

Aim/Objective:

To learn and practice the fundamental Linux commands used for file and directory management, navigation, user interaction, permissions, process management, and system administration in a Linux environment..

Description:

Linux is a powerful, open-source operating system widely used in servers, cloud computing, DevOps, and cloud-native application development. The Linux Command-Line Interface (CLI) enables users to interact directly with the operating system through commands, offering greater flexibility and efficiency than graphical interfaces..

Pre-Requisites:

- A Linux user account with terminal access.
- Basic understanding of directories, files, and folder hierarchy.
- What is a Shell?
- How commands can be executed (through Shell)?

Pre-Lab:

1. What's Linux? Explain four major parts.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Shell Versus Terminal

3. Give 2 examples for the Structure of Commands. Linux command typically consists of a program name followed by options and arguments

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. Practice essential commands used to interact with the operating system(LINUX) via the command-line interface (CLI)

BASIC COMMANDS	FUNCTIONALITY
Man	
cal	
date	
who	
ls	
nano	
cat	

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In-Lab:

1. How to Login into the KLU Linux server using putty.

2. How to Compile and run C Programs with GCC

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Define process

4. Define system call

5. Write brief description and prototypes in the space given below for the following process subsystem system call Ex: -—$man 2 <system call name>‖fork(), getpid(), getppid(), _exit(), wait(), nanosleep(), waitpid(), execve()

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Data and Results:

Analysis and inferences:

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

POST LAB

1. Practice essential commands used to interact with the operating system(LINUX) via the command-line interface (CLI):

BASIC COMMANDS	FUNCTIONALITY
rm	
rmdir	
mv	
sort	
grep	
echo	

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

chmod	
head	
tar	

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Data and Results:

Analysis and inferences:

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. What are the three main purposes of an operating system?

2. What is the kernel of an OS?

3. What is the purpose of a system call in an operating system?

4. What do you mean by file operations? List some commands

5. What is a process?

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: Basic file and process subsystem system calls

Aim/Objective:

To understand and implement UNIX system programs using basic file and process subsystem system calls for file handling and process management.

Description:
This experiment introduces students to the fundamentals of UNIX system programming by exploring the basic file and process subsystem system calls provided by the operating system kernel. Students will learn how to perform file operations such as creating, opening, reading, writing, repositioning, and closing files using low-level system calls

Pre-Requisites:

- Basic knowledge of the UNIX/Linux operating system and its command-line interface.
- Familiarity with the C programming language, including functions, pointers, and file handling concepts.
- Understanding of the compilation process using the **GCC** compiler.
- A UNIX/Linux environment (Ubuntu, Fedora, CentOS, or equivalent) with terminal access.

Pre-lab task:

1. Describe how library functions differ from system calls

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Describe how to handle Errors from System Calls and Library Functions.

3. List Standard file descriptors, File access modes

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. Write brief description and prototypes in the space given below for the following file subsystem system calls Ex: -—$man 2 <system call name>‖ These system calls can be used to perform I/O on all types of files: open(), read(), write(), close(), lseek()

5. Define file offset

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In lab task

1. write a Unix system Program that does I/O (io.c). It create a file (./myfile) that contains the string —hello world‖. To accomplish this task, the program makes three calls into the operating system. The first, a call to open(), opens the file and creates it; the second, write(), writes some data to the file; the third, close(), simply closes the file thus indicating the program won‘t be writing any more data to it. These system calls are routed to the part of the operating system called the file system, which then handles the requests and returns.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Write a program (fork.c) to demonstrate a simple fork showing PID, PPID in both parent and child

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

- **Data and Result:**

- **Analysis and Inferences:**

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Post Lab

1. Write brief description and prototypes in the space given below for the following libary calls
   Ex: -—$man 3 <library function name>‖exec family of library functions: execl(), execv(), execlp(), execvp(), sleep(), pause(), exit(), printf(), scanf(), strdup()

   Ans.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions:

1. **Explain the concept of a process and mark its differences from a program.**

2. **What are the steps performed by an OS to create a new process?**

3. **What is the key difference between a trap and an interrupt?**

4. **Consider the following piece of C code:**

```
void main( ) {
fork( );
fork( );
exit( );
}
```

How many child processes are created upon execution of this program?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

5 To a programmer, a system call looks like any other call to a library procedure. Is it important that a programmer know which library procedures result in system calls? Under what circumstances and why? For each of the followingsystemcalls,give a condition that causes it to fail: open, close, and lseek. can the count = write(fd, buffer, nbytes); call return any value in count other than nbytes?

If so, why?A file whose file descriptor is fd contains the following
sequence of bytes: 2, 7, 1, 8, 2, 8, 1, 8, 2, 8, 4.
The following system calls are made: lseek(fd, 3, SEEK SET); read(fd, &buffer, 4);
where the lseek call makes a seek to byte 3 of the file. What does buffer contain after the read has completed?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions:

6. If you open a file for read–write with the append flag, can you still read from anywhere in the file using lseek? Can you use lseek to replace existing data in the file?

Evaluator Remark (if any):	**Marks Secured _ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: System calls

Aim/Objective:

To understand and implement the use of the **fork()**, **execve()**, **wait()**, and **exit()** system calls in a UNIX/Linux
environment for creating and managing processes, and to demonstrate how these system calls are used in the implementation of a simple shell.

Description:
In UNIX/Linux operating systems, a shell acts as a command interpreter that accepts user commands and executes
them by creating new processes. This experiment demonstrates the use of four fundamental process management
system calls: fork(), execve(), wait(), and exit().

Pre-Requisites:

- Basic knowledge of the UNIX/Linux operating system.
- Familiarity with the Linux command-line interface (CLI) and shell commands.
- Understanding of C programming fundamentals, including functions, pointers, and arrays.
- Knowledge of compiling and executing C programs using the **GCC** compiler.

Pre-Lab:

1. Provide an overview of how fork(), exit(), wait(), and execve() are commonly used together in shell. (This diagram outlines the steps taken by the shell in executing a command: the shell continuously executes a loop that reads a command, performs various processing on it, and then forks a child process to exec the command.)

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Describe the value returned in the status argument of wait() and waitpid()

3. Compar the process creation using fork(), vfork(), and clone().

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In Lab:

1.The shell is just a user program. It shows you a prompt and then waits for you to type something into it. You then type a command (i.e., the name of an executable program, plus any rguments) into it; in most cases, the shell then figures out where in the file system the executable resides, calls fork() to create a new child process to run the command, calls some variant of exec() to run the command, and then waits for the command to complete by calling wait(). When the child completes, the shell returns from wait() and prints out a prompt again, ready for your next command.

The separation of fork() and exec() allows the shell to do a whole bunch of useful things rather easily. For example:

$ wc p3.c > newfile.txt

In the example above, the output of the program wc is redirected into the output file newfile.txt (the greater-than sign is how said redirection is indicated). The way the shell accomplishes this task is quite simple: when the child is created, before calling exec(), the shell (specifically, the code executed in the child process) closes standard output and opens the file newfile.txt. By doing so, any output from the soonto be-running program wc is sent to the file instead of the screen (open file descriptors are kept open across the exec() call, thus enabling this behavior.

Your Program should do exactly this. The reason this redirection works is due to an assumption about how the operating system manages file descriptors. Specifically, UNIX systems start looking for free file descriptors at zero. In this case, STDOUT FILENO will be the first available one and thus get assigned when open() is called. Subsequent writes by the child process to the standard output file descriptor, for example by routines such as printf(), will then be routed transparently to the newly-opened file instead of the screen.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

1) (a) Write a Program (p4.c) that creates new process. Child exec command "wc" and redirect standard output to a file(p4.output) by doing close(STDOUT_FILENO); open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);child exec and run a UNIX command "wc" to Count Lines, Words & Characters. Output from the soonto be-running program wc should be sent to the file(p4.output) instead of the screen(standard output file descriptor). Parent waits till child redirect and exec "wc".

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

1(b) Interchange wait() with waitpid() as the call wait(&wstatus) is equivalent to: waitpid(-1, &wstatus, 0);

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Wrie a program (child_status.c) to demonstrate the use of wait() and the W* macros for analyzing the child status returned by wait(). The parent process calls wait() to delay its execution until the child finishes execution with termination or exit status. Parent uses wait to obtain child's termination status and output on screen. The WEXITSTATUS macro fetches the exit status

POST LAB

1. Write a Program (orphan.c) that creates an orphan by letting child sleep for 2 minutes parent doesn't call wait and dies immediately. Display the pid of the process that adopted orphan.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Write a Program (zombie.c) that creates an zombie by letting parent sleep & child dies. Execute the program in background. See that ps(1) displays the string <defunct> to indicate a process in the zombie state.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Define Daemon

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

- **Data and Results:**

- **Analysis and Inferences:**

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. When an interrupt or a system call transfers control to the operating system, a kernel stack area separate from the stack of the interrupted process is generally used. Why?

2. List some daemons active on your system, and identify the function of each one.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. On an Intel x86 system under Linux, if we execute the program that prints __hello, world‘‘ and do not call exit or return, the termination status of the program — which we can examine with the shell—is 13. Why?

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

ExperimentTitle : Multithreading programs

Aim/Objective:

To understand and apply the **POSIX Threads (Pthreads) API** for developing basic multithreaded programs in C, including thread creation, execution, synchronization, and termination, in order to perform concurrent tasks efficiently and gain practical experience in parallel programming concepts.

Description:

This experiment introduces the fundamentals of multithreaded programming using the POSIX Threads (Pthreads) API in the C programming language. Students learn how to create, manage, synchronize, and terminate multiple threads within a single process. The experiment covers essential Pthreads functions such as pthread_create(), pthread_join(), pthread_exit(), and basic synchronization mechanisms like mutexes to ensure safe access to shared resources. **Pre-**

Requisites:

- Basic knowledge of the **C programming language**.
- Understanding of **UNIX/Linux operating system** concepts.
- Familiarity with the **Linux command-line environment** and GCC compiler.
- Knowledge of **processes and threads** and their differences.
- Understanding of basic **process management** concepts.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Pre-Lab:

1. Give Comparison of process and thread primitives

2. How to Compile Threaded Programs

3. How to Create and Terminate Threads Routines

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. There are several ways in which a Pthread may be terminated, list them

5. Explain Joining and Detaching Threads

6. Write a program (pthread-hello.c) that creates 5 threads with the pthread_create() routine. Each thread prints a "Hello World!" message, and then terminates with a call to pthread_exit().

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In-Lab:

1. Write a pthread program(thread_create_simple_args.c) illustrating Simpler Argument Passing to a Thread and Return values from the thread to the parent. Parent Waits for Thread Completion using pthread_join and get exit status from the new thread.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. (a) Write the main program (threads-counter.c) creates two threads using Pthread create(). Each thread starts running in a routine called worker(), in which it simply increments a counter in a loop for loops number of times. The value of loops determines how many times each of the two workers will increment the shared counter in a loop.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2(b) When the program is run with the value of loops set to 1000, what do you expect the final value of counter to be?when we gave an input value of 10,00,000, instead of getting a final value of 20,00,000, we instead get what value? So why is this happening?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

- **Data and Result:**

- **Analysis and Inferences:**

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

POST LAB

1. Write a pthread program(thrd-summation.c) illustrating how to create a simple thread and some of the pthread API. This program implements the summation function where the summation operation is run as a separate thread.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. List reasons why a mode switch between threads may be cheaper than a mode switch between processes.

2. How is a thread different from a process?

3. List some advantages and disadvantages of using kernel-level threads.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. What are two differences between user-level threads and kernel-level threads? Under what circumstances is one type better than the other?

5. What resources are used when a thread is created? How do they differ from those used when a process is created?

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: CPU scheduling algorithms

Aim/Objective:

To design and implement a userspace CPU scheduling simulator that models the behavior of **First Come First Serve (FCFS)**, **Shortest Job First (SJF)**, and **Round Robin (RR)** scheduling algorithms using a synthetic workload, and to evaluate their performance based on scheduling metrics such as waiting time, turnaround time, response time, and CPU utilization.

Description:

This experiment focuses on developing a userspace simulator that emulates the process scheduling functionality of an operating system. The simulator generates a synthetic set of processes with attributes such as arrival time and burst time, and executes them using FCFS, SJF, and Round Robin scheduling algorithms.

Pre-Requisites:

- Basic knowledge of **Operating System** concepts and process management.
- Understanding of **CPU scheduling algorithms** such as FCFS, SJF, and Round Robin.
- Basic understanding of **process attributes**, including arrival time, burst time, waiting time,turnaround time, and response time.
- Experience with the **Linux/UNIX environment** and terminal commands.

Pre-Lab Task:

1. When to Schedule

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Describe Scheduling criteria

3. Explain preemptive and nonpreemptive process scheduling

4. Write a C program(fcfs-arrival.c) to implement FCFS process scheduling algorithm by considering arrival times.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In-Lab

1. Write a C program to implement SJF process scheduling algorithm by considering arrival times((sjf-arrival.c).

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Post – Lab:

1. Write a C program(rr-queue-arrival.c) to implement Round Robin process scheduling algorithm using queue by considering arrival times.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Data and Results

Analysis and inferences:

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions:

1. What is time slicing?

2. Describe the round-robin scheduling technique.

3. What does it mean to preempt a process?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. Explain the difference between preemptive and nonpreemptive scheduling.

5. Define Process Priorities (Nice Values)

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

6. Define CPU Affinity

7. What are POSIX realtime scheduling extensions?

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: Use of the Linux **Proc File System**

Aim/Objective:

To demonstrate the use of the Linux **Proc File System (/proc)** for examining a running process by analyzing its virtual memory map through **/proc/<pid>/maps** and retrieving process status information from **/proc/<pid>/status**.

Description:

In this experiment, the **/proc/<pid>/maps** file is used to examine the virtual memory layout of a running process. It lists all memory regions allocated to the process, including the executable code (text segment), initialized and uninitialized data, heap, stack, shared libraries, and memory-mapped files, along with their memory addresses and access permissions..

Pre-Requisites:

- Basic knowledge of the Linux operating system and command-line interface.
- Understanding of processes, Process IDs (PIDs), and process lifecycle.

Pre-Lab:

1. Draw typical memory layout of a process on Linux/x86-32

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Define Stack, Stack Frames, user stack, kernel stack.

3. Give an example of a process stack

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. Write a program (va.c) that prints out the locations of the main() routine (where code lives), the value of a heap-allocated value returned from malloc(), and the location of an integer on the stack.

5. Define Static and Shared Libraries

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In-Lab:

1. Understand /proc pseudo-filesystem, type command line $ man 5 proc, list selected files in each /proc/PID directory

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. what is the purpose of systemd process, give its pid? The parent of any process can be found by looking at the Ppid field provided in the Linux-specific /proc/PID/status file.

3. Among the files in each /proc/PID/ directory is one named status, which provides a range of information about the process. Execute the given command line: $ cat /proc/1/status. Understand and write description for important fields given in output.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. List Selected files in /proc/PID directory

5. Find out how much memory a process currently has locked by inspecting the VmLck entry of the Linux-specific /proc/PID/status file.

6. Find the credentials of a process by examining the Uid, Gid, and Groups lines provided in the Linux-specific /proc/PID/status file

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

7. View Various general kernel settings under /proc/sys/kernel/. Some /proc files can be read only by the file owner (or by a privileged process).The Linux kernel limits process IDs to being less than or equal to 2^22. Among the Various general kernel settings, which file gives the upper limit for process IDs? Try # cat /proc/sys/kernel/pid_max This limit is adjustable via the value # echo 100000000 > /proc/sys/kernel/pid_max

8. See which files a process currently has open by viewing contents of all /proc/PID/fd entries which contains symbolic links for each of the file descriptors currently opened by the process. Any process can access its own /proc/PID directory using the symbolic link /proc/self.

9. Displays the number of the CPU on which a process is currently executing or last executed Example: $ cat /proc/PID/stat | awk '{print $39}'

10. Find out which shared libraries a process is currently using, list the contents of the corresponding Linux-specific /proc/PID/maps file

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

11. See the location of the shared memory segments and shared libraries mapped by a program Using the Linux-specific /proc/PID/maps file.

12. Draw a memory layout of a process showing Locations of shared memory, memory mappings, and shared libraries (x86-32)

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Data and Results :

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Analysis and inferences:

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Post-Lab:

1. Write a program to display locations of program variables in process memory segments. Give comments to indicate which memory segments each type of variable is allocated in

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Analysis and Inferences:

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. What are the elements of a process image?

2. The /proc file system exposes a range of kernel information to application programs. What are they?

3. In which manual page, information about the /proc file system can be found?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. How to list all processes that have a particular file pathname open?

Evaluator Remark (if any):	**Marks Secured ___out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: System V Shared Memory

Aim/Objective:

To implement and demonstrate System V Shared Memory and shared file mapping mechanisms for efficient data sharing and communication between processes in a UNIX/Linux environment.

Description:

This experiment demonstrates the use of **System V Shared Memory** as an Interprocess Communication (IPC) mechanism in UNIX/Linux systems. A shared memory segment is created using System V IPC system calls, allowing multiple processes to access and exchange data through a common memory area. One process writes data into the shared memory, while another process reads and processes the same data, illustrating fast and efficient communication without relying on file-based I/O.

Pre-Requisites:

- Basic concepts of **UNIX/Linux operating systems**.
- Fundamentals of **processes** and **process management**.
- Concepts of **Interprocess Communication (IPC)**.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Pre-Lab:

1. Draw a example of a memory-mapped file. Open file (fd) is mapped into the address space of the calling process. The mapped memory being somewhere between the heap and the stack.

2. Linux, like all modern UNIX implementations, provides a rich set of mechanisms for interprocess communication (IPC), list them

3. Illustrate Two processes with a shared mapping of the same region of a file

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In - Lab Task:

Write three programs:

1. Write a program to creates a shared memory segment.

.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Write a program to attaches the shared memory segments identified by its command-line arguments.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Write a program to deletes the shared memory segments identified by its command-line arguments

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Post Lab:

1. Write a program(t_mmap.c) using mmap() to create a shared file mapping

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Write a program (pstack.c) that stores a persistent stack in file ps.img, which begins life as a bag of zeros, e.g., created on the command line via the truncate or dd utility. The file contains a count of the size of the stack and an array of integers holding stack contents. After mmap()-ing the backing filewe can access the stack using C pointers to the in-memory image, e.g., p->n accesses the number of items on the stack, and p->stack the array of integers. Because the stack is persistent, data push‘d by one invocation of pstack can be pop‘d by the next.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Give Purposes of various types of memory mappings

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. What characteristics are observed while going up the memory hierarchy?

2. What is relocation of a program?

3. How does the use of virtual memory improve system utilization? which system call creates a new memory mapping in the calling process‘s virtual address space.

4. What are the two models of interprocess communication? What are the strengths and weaknesses of the two approaches?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

5. When an interrupt or a system call transfers control to the operating system, a kernel stack area separate from the stack of the interrupted process is generally used. Why?

6. To create a file mapping, we perform 2 steps. What are they?

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: Perform read/write throughput across direct I/O, buffered I/O, and memory-mapped I/O.
Aim/Objective:

To perform read and write throughput analysis using **direct I/O, buffered I/O, and memory-mapped I/O**, and compare their performance in terms of speed, memory usage, and I/O efficiency.

Description:

This experiment focuses on evaluating the performance of three file I/O mechanisms available in UNIX/Linux systems: **buffered I/O**, **direct I/O**, and **memory-mapped I/O (mmap)**. Buffered I/O uses the operating system's page cache to improve performance through caching, while direct I/O transfers data directly between user space and storage, bypassing the page cache. Memory-mapped I/O maps a file into a process's virtual address space, enabling file access through normal memory operations.

Prerequisite:

- Basic understanding of the UNIX/Linux operating system and command-line environment.
- Knowledge of file systems and file input/output (I/O) concepts.
- Familiarity with C programming and system calls.
- Understanding of file descriptors and file access modes.

Pre-Lab Task:

1. Explain Kernel Buffering of File I/O: The Buffer Cache

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Illustrate I/O Buffering: Provide an overview of the buffering employed (for output files) by the stdio library and the kernel, along with the mechanisms for controlling each type of buffering.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Explain System calls for controlling kernel buffering of file I/O:fsync(), fdatasync(), sync()

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In Lab Task:

1. Write a program(direct_read.c) Using O_DIRECT while opening a file for reading to bypass the buffer cache for direct I/O

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Write a program to demostrate fsync()

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Data and Results

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Analysis and Inferences

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Post Lab:

1. Explain Buffering in the stdio Library

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Explain Flushing a stdio buffer. Give prototype for fflush() library function.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Using open() system call and O_SYNC flag, how to Make all writes synchronous, give code snippet.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. What does a return value of 0 from printf mean?

2. Which Linux-specific open() flag allows specialized applications to bypass the buffer cache.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. When working with disk files, the read() and write() system calls don‘t directly initiate disk access. Why?

4. List C library I/O functions

.

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: Write program to read directories, uses stat() to retrieve information . Demostrate the file subsystem system calls and file commands

Aim/Objective:

To develop a UNIX/Linux program that reads directory contents and uses the stat() system call to retrieve detailed information about files and directories. The experiment also aims to demonstrate the use of file subsystem system calls and common file management commands for creating, accessing, modifying, and managing files in the UNIX/Linux environment.

Description:

This experiment focuses on understanding the UNIX/Linux file subsystem by exploring directory handling, file information retrieval, and file management operations. A program is developed to read the contents of a specified directory and use the `stat()` system call to obtain detailed information about each file and subdirectory, including file type, size, permissions, owner, group, and timestamps.

Prerequisite:

- Basic understanding of the UNIX/Linux operating system and command-line interface.
- Familiarity with the C programming language, including functions, pointers, and file handling.
- Knowledge of the UNIX/Linux file system hierarchy, files, and directories.

Pre-Lab Task:

1. Essential commands used to interact with the operating system(LINUX) via the command-line interface (CLI):mount, strace, umount, ln, stat, ln -s, ls -i, ls -l

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Write brief description and prototypes in the space given below for the following file subsystem system call
Ex: -—$man <system call name>‖ stat(), opendir(), readdir(), closedir(), rmdir(), link(), unlink(), rename()

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In Lab Task:

1. Write a program(t_stat.c) that uses stat() to retrieve information about the file named on its command line.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Write a program(lsdir.c) to read directories.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Data and Results

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Analysis and Inferences:

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Post Lab:

1. List the information typically found in, or associated with, the file inode. Give the stat structure declaration.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. List the contents of the Simplified Ext2 Inode

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Give structure dirent declaration: format of directory entries

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. Hard Links vs Soft Links

.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

5. Explain Linux Permission Bits and access control list (ACL) per directory

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. Do you expect any of a file‘s three timestamps to be changed by the stat() system call? If not, explain why.

2. What is the difference between a file and a database?

3. What is a pathname? State the two alternate ways to assign pathnames.

4. What is an inode in UNIX?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

5. Explain the purpose of the open() and close() operations.

6. Verify on your system that the directories dot and dot-dot are not the same, except in the root directory

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: Develop thread-safe bounded queue using mutex and condition variables.

Aim/Objective:

To develop and implement a **thread-safe bounded queue** using **mutex locks** and **condition variables**, enabling multiple producer and consumer threads to safely share a fixed-size queue while ensuring proper synchronization, preventing race conditions, and handling queue full and empty conditions efficiently.

Description:

A thread-safe bounded queue is a synchronized data structure that allows multiple producer and consumer threads to access a shared queue concurrently without causing data corruption or race conditions. The queue has a fixed capacity (bounded), preventing unlimited insertion of elements. Mutex locks are used to ensure mutual exclusion while accessing the shared queue, and condition variables are used to make threads wait efficiently when the queue is full (for producers) or empty (for consumers).

Prerequisite:

- Basic knowledge of **C programming**.
- Understanding of **processes and threads** in operating systems.
- Familiarity with **POSIX Threads (Pthreads)** library.
- Knowledge of **mutexes (pthread_mutex_t)** and **condition variables (pthread_cond_t)**.

Pre-Lab Task:

1. Define critical section.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Describe two tools that threads can use to synchronize their actions: mutexes and condition variables.

3. Illustrate how to use a mutex to protect a critical section?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. Search for all mutex-related man pages. hint: $man -k pthread_mutex

5. Write brief description and prototypes in the space given below for the following POSIX fuctions
Ex: -—$man 3p <system call name>‖ pthread_mutex_lock(), pthread_mutex_unlock(), pthread_mutex_ trylock(), pthread_cond_init(), pthread_cond_signal() , pthread_cond_broadcast(), pthread_cond_wait(), pthread_cond_timedwait(), int pthread_mutex_init(), int pthread_mutex_destroy()

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

6. Explain Statically Allocated Mutex and Dynamically Initializing a Mutex

7. Describe Statically Allocated and Dynamically Allocated Condition Variable

8. Show A deadlock when two threads lock two mutexes

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In Lab Task:

1. Write a program prod_condvar.c that implement POSIX threads producer-consumer problem using a condition variable. The producer code should call pthread_mutex_unlock(), and then call pthread_cond_signal(). The (consumer) thread should use pthread_cond_wait().

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Write a program prod_no_condvar.c to implement POSIX threads producer-consumer problem that doesn't use a condition variable but use a mutex-protected variable, avail, to represent the number of produced units awaiting consumption.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Data and Results

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Analysis and Inferences:

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Post Lab:

1. Write a program (thread_incr_mutex.c) that employs two POSIX threads to increment the same global variable, synchronizing their access using a mutex. As a consequence, updates are not lost.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. A condition variable always has an associated mutex. Both of these objects are passed as arguments to pthread_cond_wait(), Explain the 3 steps performed by both that signify natural association.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. Is it possible to have concurrency but not parallelism? Explain.

2. What is a race condition?

3. Provide three programming examples in which multithreading provides better performance than a single- threaded solution.

4. Describe two kernel data structures in which race conditions are possible. Be sure to include a description of how a race condition can occur.

5. Explain the differences between deadlock, livelock, and starvation.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

6. Which sequence of steps is correct?

1. Lock a mutex (pthread_mutex_lock).
2. Change the condition protected by the mutex.
3. Signal threads waiting on the condition (pthread_cond_broadcast).
4. Unlock the mutex (pthread_mutex_unlock).

or

1. Lock a mutex (pthread_mutex_lock).
2. Change the condition protected by the mutex.
3. Unlock the mutex (pthread_mutex_unlock).
4. Signal threads waiting on the condition (pthread_cond_broadcast).

Define thread-safe

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: Solve Classic synchronization problems. Introduce a race and detect it.

Aim/Objective: To understand process and thread synchronization by implementing classic synchronization problems using synchronization primitives, and to demonstrate the occurrence of race conditions by intentionally introducing shared resource conflicts and detecting them using appropriate debugging and analysis techniques.

Description:

This experiment focuses on synchronization mechanisms used in concurrent programming. Students implement classic synchronization problems such as the Producer–Consumer, Reader–Writer, and Dining Philosophers problems using mutexes, semaphores, or condition variables to ensure correct coordination among multiple threads or processes.

Prerequisite:

- Basic knowledge of operating system concepts and process/thread management.
- Understanding of concurrency and shared memory.
- Familiarity with synchronization primitives such as mutexes, semaphores, and condition variables.
- Knowledge of thread creation using POSIX Threads (Pthreads).

Pre-Lab Task:

1. Describe Race Condition

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Define semaphore

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Describe Counting Semaphores & Binary Semaphores

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. Illustrate how to use a semaphore to synchronize two processes

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

5. Describe POSIX IPC facilities (message queues, semaphores, and shared memory)

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In lab task

1. Give solution to the dining philosophers problem that is deadlock-free using semaphores. It uses an array, state, to keep track of whether a philosopher is currently eating, thinking, or hungry (trying to acquire forks). A philosopher may move into eating state only if neither neighbor is eating. The program uses an array of semaphores, one per philosopher, and use procedures, take_forks(), put_forks(), and test().

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Data and Results

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Analysis and Inferences

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

1. Give solution to the readers and writers problem using Semaphores.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Post Lab Task:

1. Write a program to introduce a race and detect it with ThreadSanitizer.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. Differentiate between Deadlock Prevention and Deadlock Avoidance.

2. What operations can be performed on a semaphore?

3. What is the difference between binary and general semaphores?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. What is the key difference between a mutex and a binary semaphore?

5. Explain how a Web browser can utilize the concept of threads to improve performance.

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: Write bpftrace one-liners to (a) count system calls per process, (b) measure block I/O latency distribution, (c) trace which processes are causing page faults; the live-debugging.

Aim/Objective: To develop practical skills in Linux kernel observability using **bpftrace** by writing one-line tracing programs to monitor system behavior, including counting system calls per process, measuring block I/O latency distributions, tracing processes responsible for page faults, and performing live kernel debugging with minimal system overhead.

Description:

This experiment introduces the use of **bpftrace**, a high-level tracing language built on **eBPF (Extended Berkeley Packet Filter)**, for dynamic kernel and application observability. Students will write and execute bpftrace one-liners to monitor real-time system activities without modifying kernel code. The tasks include counting system calls made by each process, measuring and visualizing block I/O latency distributions, identifying processes that generate page faults, and performing live debugging of kernel events. Through these exercises, students gain hands-on experience in tracing, performance analysis, troubleshooting, and understanding Linux kernel behavior with low-overhead, production-safe instrumentation.

Prerequisite:

- Basic knowledge of the Linux operating system and command-line interface.
- Understanding of operating system concepts such as processes, system calls, virtual memory, and file systems.
- Familiarity with kernel events and basic performance monitoring concepts.

Pre-Lab Task:

1. What is bpftrace? On which operating system it is used? What technology does bpftrace use for tracing? Which compiler backend is used by bpftrace?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Differentiate between tracepoints and kprobes.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Write the bpftrace command to print "hello world" using the BEGIN probe.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4.What is the purpose of the printf() function in bpftrace?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

5.Define eBPF Virtual Machine

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

6.Explain bpf() system call.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

7.llustrate How a userspace program interacts with eBPF programs and maps in the kernel using syscalls

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In lab task

1. Write bpftrace one-liners to (a) count system calls per process, (b) measure block I/O latency distribution, (c) trace which processes are causing page faults

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Data and Results

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Analysis and Inferences

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Post lab task

1. Describe how bpftrace traces file-open system calls.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Write a bpftrace one-liner to display the names of files opened by processes.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Describe how kernel stack profiling works in bpftrace.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

4. Apply stratce for a sample program

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. Analyze the role of bpftrace in Linux observability and explain how it differs from traditional logging.

2. Compare tracepoints and kprobes

3. How can file-open tracing help detect unauthorized file access?

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment Title: Apply bpftrace as the awk-of-the-kernel: one-liners that trace system calls, scheduler events, file opens, network packets, page faults.

Aim/Objective: To apply **bpftrace** as the "AWK of the Linux kernel" for dynamic tracing by writing and executing one-line tracing programs to monitor kernel events such as system calls, scheduler activities, file operations, network packet processing, and page faults, thereby analyzing system behavior and performance in real time with minimal overhead.

Description:

This experiment introduces **bpftrace**, a high-level tracing language built on **eBPF (Extended Berkeley Packet Filter)**, for dynamic analysis of Linux kernel and user-space applications. Students will learn to write and execute concise one-line tracing scripts to observe kernel activities without modifying application code or recompiling the kernel. The experiment covers tracing of system calls, scheduler events, file open operations, network packet activity, and page faults to gain insights into system performance, resource utilization, and application behavior. Through these practical exercises, students develop skills in Linux performance monitoring, troubleshooting, and kernel-level observability using modern eBPF-based tracing techniques.

Prerequisite:

- Basic knowledge of the Linux operating system and command-line interface.
- Understanding of Linux kernel concepts such as processes, threads, and system calls.
- Familiarity with shell scripting and common Linux utilities.
- Basic knowledge of computer architecture and operating system fundamentals.

Pre-Lab Task:

1. What is the relationship between bpftrace and eBPF?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Explain why bpftrace is often described as the "awk-of-the-kernel.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Discuss the role of aggregation functions (count(), sum(), hist()) in summarizing tracing results.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

In Lab

1. Write bpftrace as the awk-of-the-kernel: one-liners that trace system calls, scheduler events, file opens, network packets, page faults.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

POST LAB

1. Write a bpftrace one-liner to count TCP packets.

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

2. Why is eBPF preferred over packet capture tools for kernel-level tracing?

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

3. Explain how bpftrace can be used to count page faults generated by each process. Give Page-Fault Stack Trace

Experiment #	<TO BE FILLED BY STUDENT>	Student ID	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	Student Name	<TO BE FILLED BY STUDENT>

Sample VIVA-VOCE Questions (In-Lab):

1. What is the purpose of attaching probes to network stack functions?

2. Why is eBPF preferred over packet capture tools for kernel-level tracing?

3. Which probe types can be used for tracing network-related kernel events?

.

Evaluator Remark (if any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Note: Evaluator MUST ask Viva-voce before signing and posting marks for each experiment.